

Self-Attention... The way I Understand

Part 3

Part 2 Summary (Key Points)

In part 2 of this series, we covered the position encoding and the need for multiple encoder layers in the encoder block. In this section we will discuss the concepts of skip connection in the context of position encoding, the concept of multi-headed attention and the interaction between the encoder and decoder where the magic happens.

Context Aware Embedding from Context Free Vectors

Let us assume we have the input statement as “The little brown fox jumped over the lazy brown dog which is fond of sweet bun and raisin sprinkled brown bread”. The word “brown” appears in position 3, 9 and 20 in the sentence. When we represent the word “brown” as a word vector using traditional embedding tools such as Word2vec or Glove or any other, the embedded form will fail to capture the different relations the word “brown” has with different words. These techniques generate context free word embedding.

The first encoding layer attempts to generate context aware embedding from the input vector which is context free. It generates different vectors for the different instances of “brown”. Hence, the resulting Q, K and V matrices of the first encoding layer will not be accurate in their purpose. The subsequent self-attention layers take as input the context aware embeddings of the words. The Q, K, V matrices of the subsequent layers will be better in their purpose than the first set. That is why we need multiple encoder layers above the first one.

With multiple Attention layers to go through, will not that smudge out the position encoding which is done before the first Self-Attention layer? Or is there a way to preserve the position encoding till the last Self-Attention layer. The Transformer architecture preserves the position encoding using a technique called “Skip Connection”. This technique is borrowed from the famous ResNet architecture that was designed to overcome the challenges of vanishing gradients in deep neural networks.

Position encoding Example

The following table shows position encoding (PE) for “brown” derived from the context free encoding and position of the word in the sentence.

Embedding Dimension index	Context free Vector encoding for brown Glove in 10 dim	Encoding for positions (EP)			Position Encoded Vector for Brown (= Context free vector encoding + EP)		
		3	9	20	3	9	20
1	0.2	0.059964	0.724287	0.973848	0.259964	0.924287	1.173848
2	0.5	0.119712	0.99879	-0.44252	0.619712	1.49879	0.05748
3	0.1	0.17903	0.653041	-0.77276	0.27903	0.753041	-0.67276
4	0.9	0.237703	-0.09825	0.793668	1.137703	0.801751	1.693668
5	0.3	0.29552	-0.78853	0.412118	0.59552	-0.48853	0.712118
6	0.5	0.352274	-0.98913	-0.98094	0.852274	-0.48913	-0.48094
7	0.9	0.40776	-0.57548	0.033623	1.30776	0.324525	0.933623
8	0.7	0.461779	0.195547	0.965658	1.161779	0.895547	1.665658
9	0.1	0.514136	0.845133	-0.47242	0.614136	0.945133	-0.37242
10	0.3	0.564642	0.96989	-0.75099	0.864642	1.26989	-0.45099

1. We have ten dimensional feature spaces. Each row in the table is for one dimension
2. Assume the word “brown” is vectorized using Glove or Word2Vec and is as shown in “Context free Vector Encoding” column
3. All the tree instances of “brown” will have the same context free vector representation which means the relation between the word “brown” and other words is lost
4. Position encoding of the positions 3, 9, 20 using Sine and Cosine function is calculated based on the formula we discussed in part 2. Sine for even dimension index and Cosine for odd.
5. Add the “Vector Encoding” and “Position Encoding” for the 3, 9 and 10th resulting in the three position encoded vectors (PE) for Brown shown in three respective columns
6. Cosine similarity between the three positional encoded vectors of brown, differ with relative positions between the two instances
7. Thus, the word brown is represented by three position encoded vectors

Cosine Similarity between the instances of “brown”	
Without position encoding (context free encoded vector). All the three instances of “Brown” are represented by same vector.	1
With position encoding 3rd and 9 th . The word “Brown” in two different positions represented by two different vectors	0.61988
With position encoding 3rd and 20 th .	0.56648

Note:

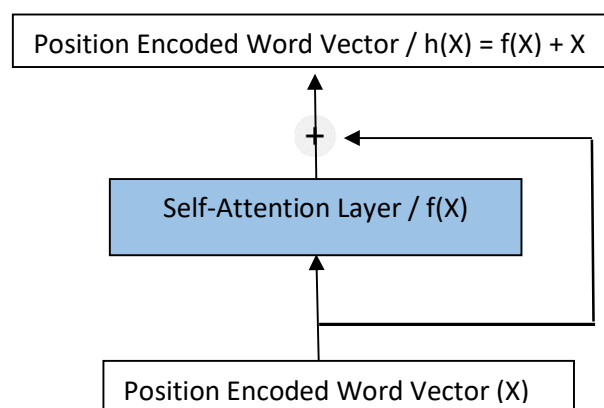
- a. Cosine distance does not meet the triangle inequality requirement for distance functions. It measures directional similarity only. Hence, two points A, B that are physically closer may have lower cosine similarity than two points A, C which are not as close.
- b. Instead, one can use Euclidian distance if the triangulation is important.
- c. The formula is modified to divide word position by 10 instead of 10000 to show the impact of position encoding
- d. The dimensionality of the encoding space is 10 in the example

Observations-

1. Cosine similarity between a word and its own self will be the max which is quantized as 1
2. A word will get different position encoded embedding when it appears in different positions in a sentence. Farther two instances are, less similar the position encoded vectors are even though word is same, indicating different interpretations for the words

Skip connections to retain position encoding

The position encoding is generated before the first encoder layer. It is carried forward through two paths. One is as a input to the next encoding layer and other through a skip connection. The idea of skip connection is borrowed from the [ResNet](#) architecture in Deep Neural Networks.



The whole idea of skip connection is to force the model to learn a function $h(X)$ in two parts, one part through the skip connection (X) and the other through the residual layer represented by $f(X) = (h(X) - X)$.

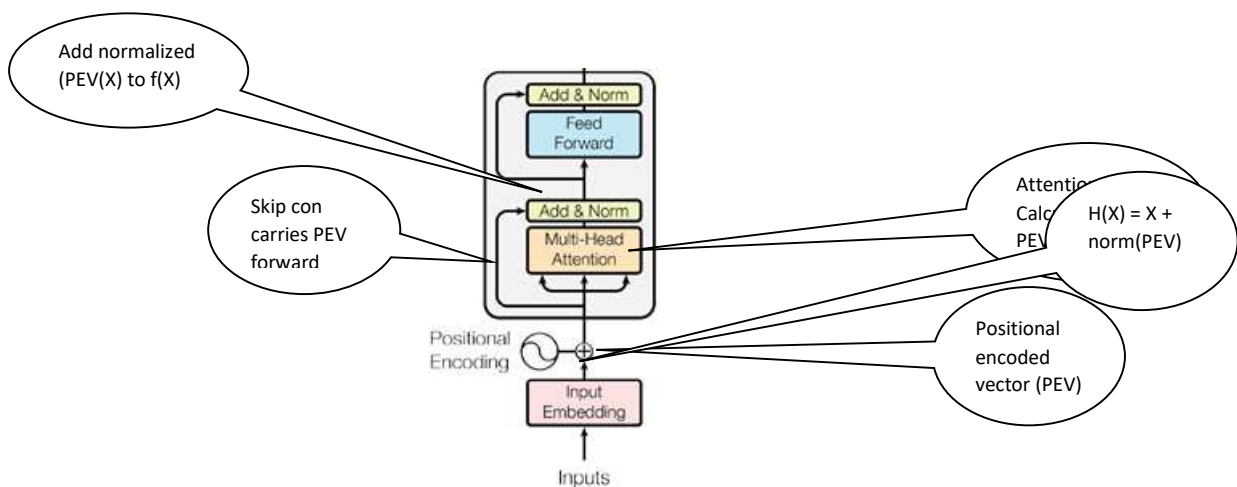


Image Source : https://www.researchgate.net/figure/The-Transformer-encoder-structure_fig1_334288604

The skip connections are also useful in the training stage where they help back propagate error gradients easily even in very deep neural network architectures without the problem of vanishing gradients. However, in this discussion, the purpose of the skip connection is also to carry the positional encoding values to the next layer as discussed above.

Multi Headed Attention (MHA)

MHA is another concept that plays an important role in the Transformer technology. The entire Transformer technology can be shown as –

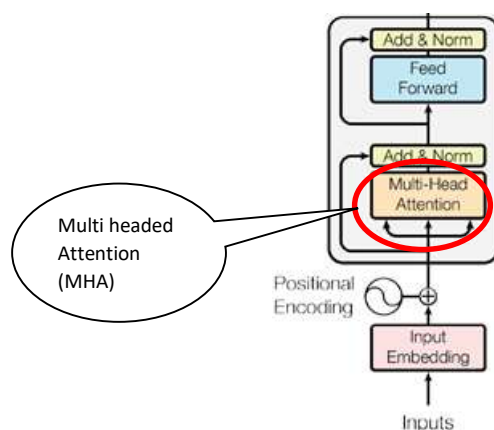
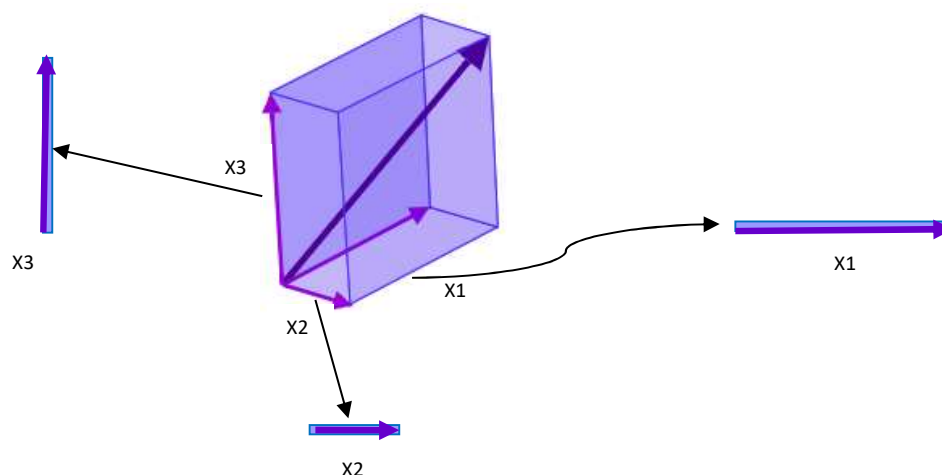


Image source: <https://cs.paperswithcode.com/method/transformer>

To understand the multi-headed attention, let us consider a 3D space with a vector as shown below. Suppose we must do some processing on this vector. Suppose that processing can be done in a lower dimensional 1D space and the result concatenated. This is like multi-headed processing. So, when we have a very high dimensional vector space, we can process the vector in lower dimensional space in parallel and concatenate the results to get back the result in original dimensional space.

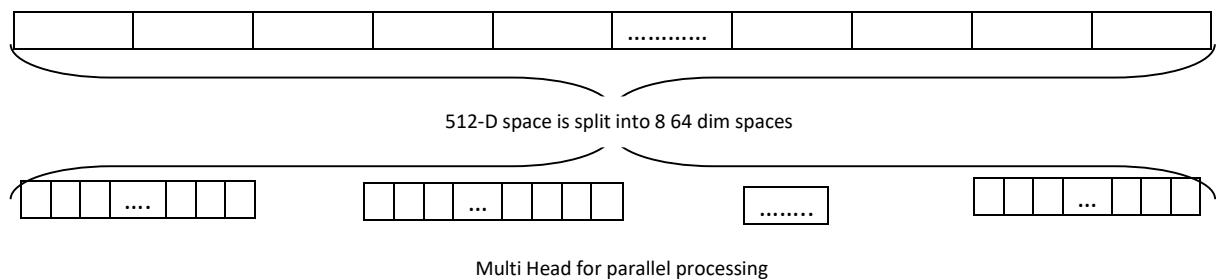


The 3d vector (shown in blue) can be analysed in three different subspaces of one dimension each and the results can be collated from all the subspaces to draw inferences about the vector in the original higher dimensional (3D) space. **These subspaces are called heads** making the process of self-

attention, multi-headed. In the above case we can say the number of heads is three. In the MHA the subspaces are disjoint.

In the original architecture by Vaswani et al, the input to the first encoder layer was a position encoded vector of 512 dimensions. Each word becomes a vector of 512-dimensional object. To find the self-attention of each word, the self-attention process must be carried out in 512 dimensions, and this can be computationally quite heavy. Further, the self-attention for each word will be in the context of all 512 dimensions put together. The problem with this is, some dimensions may overwhelm others / carry relatively larger weight than others and in the process, influence the self-attention more than others. This is like the situation where the attributes in a feature space are at different scales and the one with largest scale swamps the distance calculation between data points, Euclidian distance calculation in KMeans clustering for instance. To prevent that, we scale the data.

Instead of scaling, in MHA, the higher dimensions space is split into multiple sub spaces (eight sub space of 64 dimensions each). When we concatenate all the sub-spaces into one space, we get back the original vector in 512 dimensions.



The self-attention mechanism is applied in parallel on all the eight sixty-four dimension vector representation of a word (eight heads) the result of which is eight self-attention values for a word in the context of corresponding subspace.

This is like Bagging ensemble in the conventional machine learning model where multiple instances decide together what the prediction should be. Those who have used Bagging, would know, predictions of the ensemble are more reliable than prediction of a single model. Similarly, the self-attention derived from multiple sub spaces is more reliable than on one single high dimensional space.

These eight self-attention values will be concatenated (physically appended together). If we represent the self-attention values using character z then,

$$Z = (Z_0, Z_1, Z_2, Z_3, Z_4, Z_5, Z_6, Z_7)$$

The effect of concatenation is to get back the 512 dimensions but eight self-attention vectors for a word! Within each of the eight heads, each word vector has the three representations as Q, K and V. All the three are of dimension 64.

Add Normalization Layer

The output of each MHA sub layer is a 512 dimensional array for each word. To this array of values, the position encoded vector of same dimension from the skip connection is added and the result normalized. This layer can thus be represented as

$$\text{LayerNorm}(X + \text{Sublayer}(X))$$

where X is the set of values coming from skip connection and Sublayer(X) is the output of the MHA. The layer normalization is defined as the standard scaler function with a scale and shift factor.

$$\text{LayerNorm}(\text{Values}) = \gamma ((V - V_{\text{bar}}) / \sigma) + \beta$$

γ -> Scale factor

β -> Shift factor

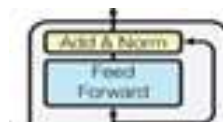
V_{bar} -> Average of Values

σ -> Standard Deviation of the Values

Feedforward Network (FFN)

This is a fully connected neural network, consists of two layers with ReLU activation function. The input and output of this layer is 512 dimensions (in the original transformer) but the layer in between (hidden layer) is of 2048 dimensions. This layer processes the input (output of the Add normalization layer) position by position. **The input data is projected from a 512 dimensional space to 2048 dimensional space. Such projections help discover the structures in the data** which can act as features during the learning stage (Deep learning methods discover features from the data). These features are projected to lower dimensions of 512 for further processing. The relation between the features learnt and the target values are stored as weights in different layers. This process of learning discovering hidden structures and using them as features is like using PCA to extract hidden correlation information in input feature space and subsequently dropping the weaker principal components to represent the extracted information in lesser dimension.

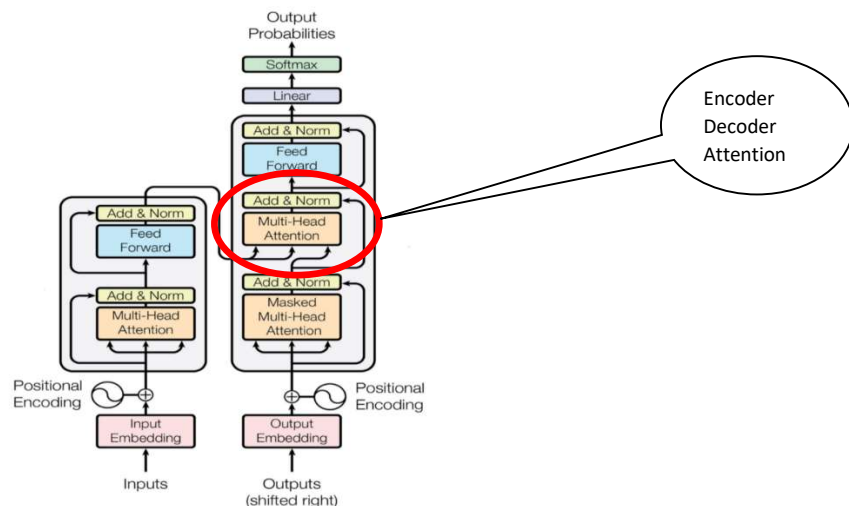
The input to FFN layer is also carried through the skip connection to be added to the features extracted by the FFN layer. To do this addition the FFN output has to be of same dimensions as the input to FFN layer. That is the reason to project the new features back into 512 dimensions.



The result of the addition goes through the normalization process to complete the processing of one Encoder cell. Multiple Encoder cells are placed in sequence to make the resulting model powerful. The output of the last Encoder layer block is sent to "Encoder-Decoder Attention" layer of all the Decoder blocks.

Decoder Block / Encoder-Decoder Attention

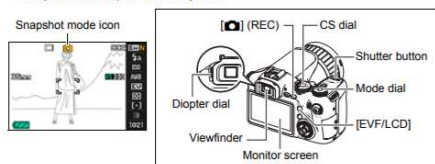
The Decoder block has similar structure as the Encoder block except for one major difference. The decoder block has a special “Encoder-Decoder Attention” layer. This is the layer where the Encoder and Decoder interact. This layer is placed next to the first “Self-Attention Layer” in the block.



The interaction happens through the Q, K and V matrices. **The Q matrix is generated through dot product between W_q and representation matrix from the previous layer of Decoder. The K and V matrices are generated through a dot product between W_k and W_v and the representation matrix from Encoder.** Thus, the **link between Encoder and Decoder is established here**. The self-attention from here on is called Encoder-Decoder Attention for obvious reasons. Why so? This is where I would like to go back to the example of the English and French manuals for the Casio digital camera.

To shoot a snapshot

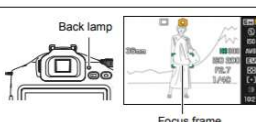
1. Remove the lens cap and then press [RECALL] (REC) to turn on the camera.
2. Set the mode dial to [Auto].
3. Set the CS dial to [Single Shot].
 - You can use either monitor screen or viewfinder to compose the image. Each press of the [EVF/LCD] button cycles through the available monitor screen and viewfinder on/off settings (page 146). If the image in the viewfinder appears blurry, use the diopter dial to adjust it.



4. Point the camera at the subject. You can zoom the image, if you want.

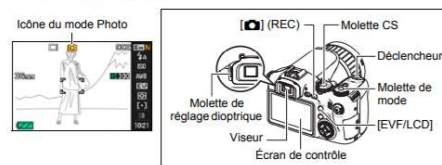


5. Half-press the shutter button to focus the image. When focus is complete, the camera will beep, the back lamp will light green, and the focus frame will turn green.



Pour prendre une photo

1. Retirez le capuchon d'objectif et appuyez sur [RECALL] (REC) pour mettre l'appareil photo en service.
2. Mettez la molette de mode en position [Auto].
3. Mettez la molette CS en position [Une seule photo].
 - Vous pouvez utiliser soit l'écran de contrôle soit le viseur pour cadrer l'image. À chaque pression du doigt sur le bouton [EVF/LCD] les réglages en et hors service de l'écran de contrôle et du viseur apparaissent à tour de rôle (page 146). Si l'image apparaît floue dans le viseur, utilisez la molette de réglage dioptrique pour ajuster la dioptrie.



4. Dirigez l'appareil photo vers le sujet. Vous pouvez faire un zoom avant ou arrière, si nécessaire.



5. Appuyez à demi sur le déclencheur pour faire la mise au point. Lorsque la mise au point est terminée, l'appareil photo bip, le témoin arrière s'allume en vert et le cadre de mise au point devient vert.

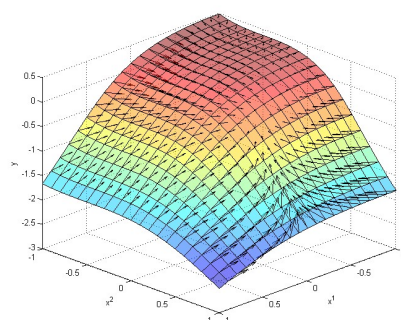


The Encoder works on the English corpus while the Decoder works on the French one. Both corpuses convey the same information with same degree of clarity. Which means the **Q, K, V based calculations for the English statements such as “how to take a snapshot” would have a corresponding Q, K, V calculations of the equivalent French statement in the first self-attention layer of the Decoder**. To establish this relation between the English and French versions, we need to tie the English and French versions of Q, K, V calculations. This is what the Encoder-Decoder Attention Layer does.

Suppose two people are trying to converse. One speaks only English while the other speaks only French. To communicate they will have to develop a language independent way to communicate. Maybe through drawing (hope both have descent drawing skills). **Similarly, the Encoder and Decoder learn to represent their respective corpuses in respective high dimensional feature space and whatever that representation maybe, I believe they will be similar in some mathematical sense as both the corpuses contain same information, same topics / dimensions except the set of tokens and the rules to use the tokens (grammar) are different.** This is also ensured by tying up the Encoder K, V and Decoder Q matrices as discussed above. During the training stage, using the back propagation, this correspondence between Encoder Decoder representation is strengthened further by learning the weights associated with the Q, K, V matrices in each layer in both Encoder and Decoder together. The weights play a critical part in playing out this almost magical result.

Weights gradient vectors, force fields, attractors for representations

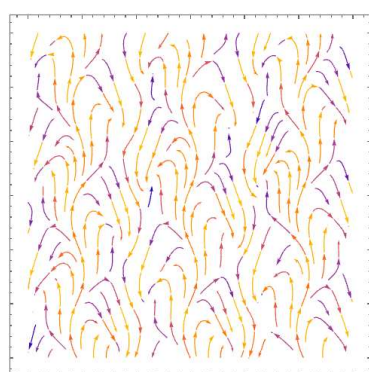
Supervised learning method based models learn from the data. They learn the relation between the dependent and independent variables. These learnings are stored in form of coefficients / weights associated with all the neurons at different layers of the model. These weights reflect the relation between input (X) and output (y) variables of a neuron in form of dy/dX (how much y changes per unit change in X and is the change positive or negative, a.k.a. gradient). This coefficient is a vector at a point indicating the direction of increase and magnitude of increase. The entire feature space is filled with infinite points and one vector at each point. For clarity, only a sample of the vectors in a feature space of three dimensions. The vectors define the hyper surface that we call model.



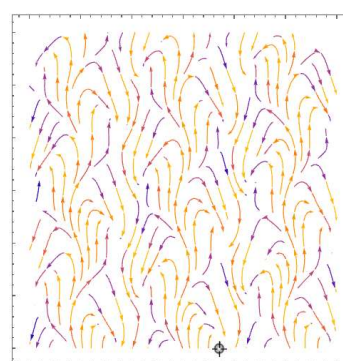
Source: https://www.researchgate.net/figure/Gradient-field-on-a-2-manifold_fig5_268572039

The differential function is also known as the force function because the vector at a point can be imagined as force acting at any object at that point. Thus, the entire feature space can be visualized as force field. At some points in this force field, the net force /vector can be zero. Such points are called attractors. The entire feature space is a landscape of multiple attractors and the force fields that form trajectories connecting the attractors. The attractors represent the different internal states a system can be in. In a trained Encoder Decoder system, these attractors are internal representations of the tokens / words generated using Self-Attention mechanism.

In this technology, all the three weight matrices put together represent the force field in the D dimensional space. The net result of the training of Encoder and Decoder together can be visualized as defining two such vector fields, one for each language. Since both the corpuses contain same information represented in same number of dimensions, the resulting vector fields in Encoder and Decoder respectively will be similar if not same. The vector fields with attractors and trajectories will crystallized during the training in the process of error reduction through back propagation.



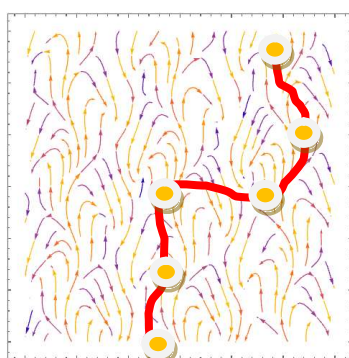
English Language Vector Field



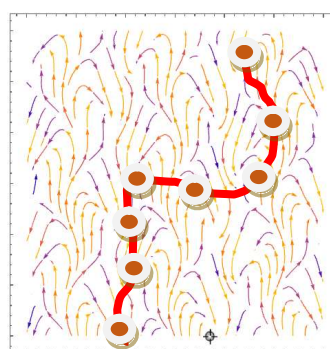
French Language Vector Field

Image Source: <https://demonstrations.wolfram.com/VectorFieldsStreamlineThroughAPoint/>

Let the input statement to the trained model be “Casio Camera is good for nature photography”. The Encoder represents this in its vector space as a trajectory passing through a set of states. The states and the trajectory are a mathematical way of representing the embedded words and their dependencies in the input sentence. The corresponding French sentence “Casio Camera est bon pour la photographie de nature” is represented by the respective states and trajectories in Decoder feature space.



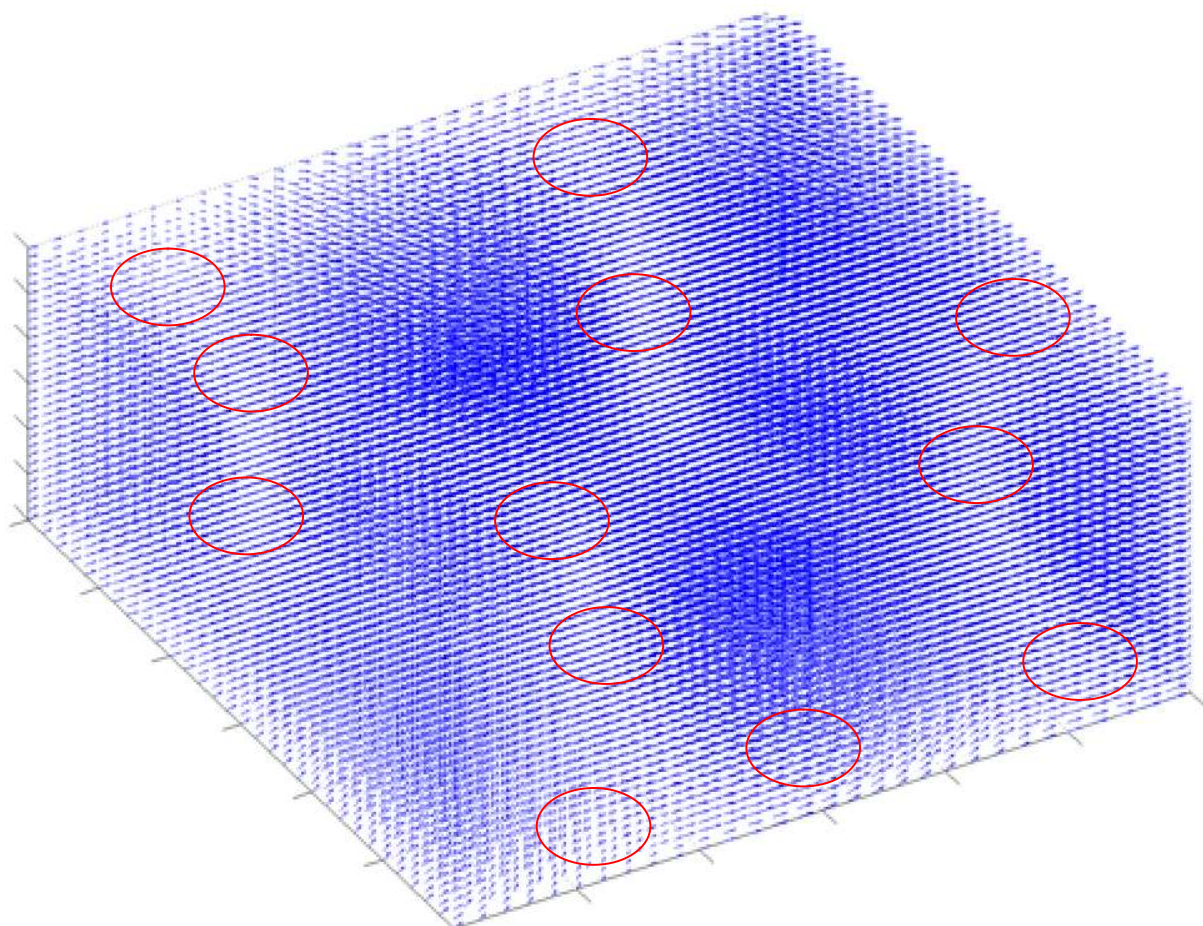
English corpus vector field



French corpus vector field

Since the two sentences communicate same information with same clarity, their trajectories in their respective vector spaces should have a one-to-one correspondence. The number of states (shown in yellow for English and maroon for French) that the respective trajectories passe through need not be the same!

When we use the trained model to translate English statement, the Encoder Key, Value representation is passed on to the Decoder which along with the French Decoder Q representation and the resulting self-attention trigger the journey through the Decoder vector space on the corresponding trajectory. **I am not saying the trajectory to be followed in Decoder will be known.** Given the self-attention value and the vector field in Decoder, the Decoder state transitions will be such that it moves from current state to next based on probability distribution across different possible states. This probability has direct relation to the underlying vector field based trajectories. In the figure below, the white patches with red circles are attractors where vectors become very small, and the different attractors are connected by paths carved out by the vectors. Also note, from one given state, there may be multiple states that can be reached and which one is reached next depends on the trajectories emanating from that state towards other states. This information can be used to calculate conditional probabilities of the subsequent states given the current state.



Source:

http://web.mit.edu/8.13/matlab/MatlabTraining_IAP_2012/AGV/DemoFiles/ScriptFiles/html/Part8_VectorFields.html

The Decoder follows the trajectory and identifies the states in sequence (time steps). These identified states are forwarded to the FFN, Linear and SoftMax layers that emit the corresponding French words in the required sequence. The number of states covered in the French vector space need not be the same as number of states in English vector field. Hence the translation is not word to word.

Note: It is impossible to discuss in 2D or 3D all that seems to happen in a very high dimensional space. The trajectories representing input statement can be highly convoluted with loops. It is left to the reader's imagination. I came across an interesting application of vector fields when working on this post. Even bees use vector fields to communicate! Check this out –

<https://www.intmath.com/blog/videos/friday-math-movie-dancing-honeybee-using-vector-calculus-to-communicate-936>

Self-admission – I would like to admit that this post and the previous ones on this topic (links on the last page) explaining how the technology works under the hood, is purely my way of understanding the same. It is based on all the information that I gathered from various sources including those that were published before this technology appeared (“Computing with Attractors”, “Geometry of Preposition Meanings”). Reading all these publications helped me understand the advantage of representing real world processes in a virtual mathematical feature space. That the weights (model parameters) learnt during training, represent spatial gradients which map the shape of the hyperspace/manifold (a.k.a. model) was a very helpful. Those points in the hyperspace where the gradients vectors become zero act as attractors and the general direction of the gradient vectors in the feature space continuum represent the trajectories connecting the attractors. The article “[Computing with Attractors](#)” by John Hertz helped me look at the RNN and LSTM as state machines where the states are represented by the attractors in a vector field. I carried these ideas forward to explore Transformers. Hope you will find it useful.

References:

1. “Attention is all you need” - <https://arxiv.org/abs/1706.03762>
2. “Getting Started with Google BERT” – Sudharsan Ravichandran / Packt
3. “Transformers for Natural Language Processing” - Denis Rothman / Packt
4. <https://jalammar.github.io/illustrated-transformer/>
5. “Python Machine Learning by Example” – Yuxi (Hayden) Liu / Packt
6. <https://erdem.pl/2021/05/understanding-positional-encoding-in-transformers>
7. https://www.scitilab.com/post_data/gradient/2019_10_17_temp_gradient/
8. http://www.sharetechnote.com/html/Calculus_Gradient.html
9. “Computing with Attractors” by John Hertz
10. “geometry-of-meaning” by Peter Gardenfors

Other articles -

1. https://www.linkedin.com/posts/mukeshrao1_transformers-and-self-attention-the-way-activity-6836637156300193792-CFtX

2. https://www.linkedin.com/posts/mukeshrao1_transformers-position-encoding-the-way-i-activity-6841316053327462400-b0xl
3. https://www.linkedin.com/posts/mukeshrao1_rnn-intuitive-way-activity-6833251434050539520-GSQR
4. https://www.linkedin.com/posts/mukeshrao1_rnn-time-steps-and-input-tensor-shape-activity-6830708878259425281-D_e8
5. https://www.linkedin.com/posts/mukeshrao1_information-noise-the-way-i-understand-activity-6760761794215735297-3MH2
6. https://www.linkedin.com/posts/mukeshrao1_adjusted-r-square-activity-6757279441057144832-3QZD
7. https://www.linkedin.com/posts/mukeshrao1_part-2-error-function-optimizers-the-activity-6754006351380389889-AF0E
8. https://www.linkedin.com/posts/mukeshrao1_error-function-convexity-optimizers-the-activity-6750400156438552577-KKQT
9. https://www.linkedin.com/posts/mukeshrao1_error-function-convexity-optimizers-the-activity-6750399758474604544-5RK1
10. https://www.linkedin.com/posts/mukeshrao1_lstm-the-way-i-understand-activity-6687006866700156928-jtfa
11. https://www.linkedin.com/posts/mukeshrao1_recurrent-neural-network-as-a-state-machine-activity-6681424681699962880-iPNp
12. https://www.linkedin.com/posts/mukeshrao1_convolution-and-pooling-conceptual-discussion-activity-6665970240704380928-QHP0
13. https://www.linkedin.com/posts/mukeshrao1_for-correct-visualisation-of-pc-and-target-activity-6663429056144510976-nsWR
14. <https://www.linkedin.com/posts/activity-6663427899460325377-6FIE>
15. <https://www.linkedin.com/feed/update/urn:li:activity:6656445261159731200>